Online Time Series Detection: Streams, Latency, and Drift

CEFET/RJ

Data Analytics Lab

dal.global@grupo.cefet-rj.br | https://eic.cefet-rj.br/dal

The Online Setting: Constraints and Objectives

Why the Setting Changes the Problem

In the offline setting, the complete series is available. In the online setting, observations arrive continuously and decisions must be made in the stream, without access to the future.

Online Detection

- Observations arrive sequentially.
- The detector decides using only what has already arrived: $\delta_t = D(x_1,\ldots,x_t)$.
- The decision must be fast enough to allow an operational response.

Online Prediction

- The goal is to predict the future occurrence of an event.
- It is an anticipatory form of event detection.
- Its usefulness depends on issuing an alert before action is no longer possible.

Necessary Balance

The method must balance speed, precision, memory, and the ability to adapt over the stream.

Three Event Scenarios

The Three Scenarios

Offline detection

Recognizing past events. The complete series is available for analysis.

Online detection

Detecting events as they happen. Decision made in the stream.

Online prediction

Anticipating events before they occur. Alert issued with horizon h.

Offline Detection — Characteristics

- The series has already been collected before analysis.
- The algorithm can look at both past and future around an event candidate.
- It is well-suited for auditing, retrospective explanation, and method benchmarking.

The offline scenario serves as a reference for understanding what is lost — and what is gained — when operating online.

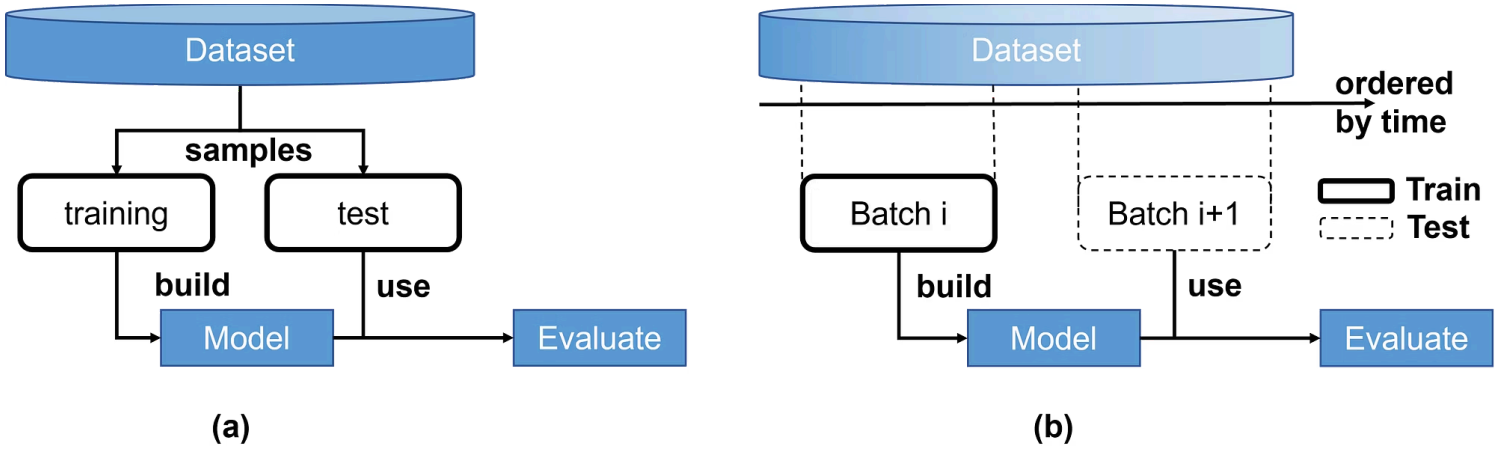
Training and testing: traditional vs streaming

Traditional scenario

- The series is split into training and test sets in a static manner.
- The model is fitted on past data and evaluated on more recent data.
- Temporal validation and hyperparameter tuning occur before deployment.

Streaming scenario

- Each batch can train or update the model for the next batch.
- Temporal order is part of the experimental protocol.
- This design mimics the operation of a system that learns from the data stream.



In streaming, batch i is used to build or update the model evaluated on batch i+1.

What is online detection

Operational definition

It is the identification of events as observations arrive, without access to the future. The detector processes the series sequentially and emits δ_t at each step.

Causality condition

$\delta_t = D(x_1, ..., x_t)$. No observation after t can be used in the decision at t.

Latency constraint

The decision must be emitted within a time budget Δt compatible with the arrival rate of the stream.

Detector output

$\delta_t \in \{0, 1\}$: 0 indicates normal behavior, 1 indicates a detected event. Some detectors also emit an anomaly score $s_t \in \mathbb{R}$.

General Online Detection Loop

Loop Structure

The detector operates in a potentially infinite loop. At each step t, it receives x_t, updates its internal state, and emits δ_t.

> $\delta_t = D(x_t; \theta_t)$

Parameters: θ_t (internal state of the detector at t), τ (decision threshold), s_t (anomaly score).

Stopping Conditions

The loop terminates when the stream ends or when an external condition interrupts monitoring.

Pseudocode — General Online Detection Loop

INPUT: stream X = $x_1, x_2, \ldots$, detector D, threshold τ
PRE-CONDITION: D initialized with state θ_0

STEP 1 — Upon receiving x_t from the stream:
    if stream ended → stop
STEP 2 — Compute anomaly score:
    $s_t \leftarrow D.score(x_t; \theta_t)$
STEP 3 — Make decision:
    $\delta_t \leftarrow 1$ if $s_t > \tau$, else 0
STEP 4 — Update internal state:
    $\theta_{t+1} \leftarrow D.update(\theta_t, x_t)$
STEP 5 — Emit δ_t (and optionally s_t)
STEP 6 — $t \leftarrow t + 1$; return to STEP 1

OUTPUT: sequence of decisions $\delta_1, \delta_2, \ldots$
POST-CONDITION: each δ_t uses only $x_1, \ldots, x_t$ (causality guaranteed)

Formalization of the Temporal Stream

Formal Definition

Let $X = \langle x_1, x_2, \ldots, x_t, \ldots \rangle$ be a potentially infinite stream of observations. At each instant t, detector D receives x_t and emits a decision $\delta_t \in \{0, 1\}$ using only the available history.

$$\delta_t = D(x_1, x_2, \ldots, x_t)$$

Parameters: $x_t \in \mathbb{R}^d$ (observation), δ_t (binary decision), t (current instant).

Causal Constraint

The detector cannot access $x_{t+1}, x_{t+2}, \ldots$ at the time of decision. This distinguishes the online scenario from the offline one, where the entire series is available.

$$\langle \ldots, x_{t-2}, x_{t-1}, x_t \mid x_{t+1}, x_{t+2}, \ldots \rangle$$

The vertical bar indicates the visibility boundary at t.

Inevitable Tension

Some events only become clear after a few observations. Using future observations violates the operational scenario. This creates a tension between immediate decision-making and reliable confirmation.

Early detection and the speed vs false positive trade-off

Definition

Detect the event as early as possible after its onset, minimizing latency $L = \hat{t} - t^*$.

> minimize $E[L]$ subject to $FPR \leq \alpha$

Parameters: L (latency), FPR (false positive rate), α (false alarm tolerance level).

Operational value

The value of an alert decays over time: $V(\text{alert}) = f(h - L)$, where h is the action horizon. In critical systems, even a small delay can be irreversible.

The central trade-off

Sensitive detector

Reacts quickly. Can anticipate important events. Tends to generate more false positives and alert fatigue.

Conservative detector

Requires more evidence. Reduces false alarms. May detect late or miss events in critical systems.

In many domains, the cost of a false negative exceeds that of a false positive — which justifies lower thresholds and earlier detection.

Fast data and the limitations of offline methods

Fast data and critical systems

Fast data involves large volume, high velocity, and change at short intervals.

- Networks, IoT, industry, healthcare, and security require continuous monitoring.
- The solution must operate under constraints of time, memory, and reliability.
- The cost of a late detection can be irreversible in critical systems.

Why offline methods don't migrate directly

- Offline methods may depend on the complete series.
- The computational cost may be incompatible with continuous data streams.
- The presence of drift changes the notion of typical behavior.
- Offline metrics may ignore the temporal delay of detection.

Static vs Dynamic Models and Memory Strategies

Static

- Trained before deployment.
- Remain unchanged during execution.
- Work well when the distribution is stable.

Dynamic

- Updated with new incoming data.
- Can track changes in the data stream.
- Require a memory policy and adaptation mechanism.

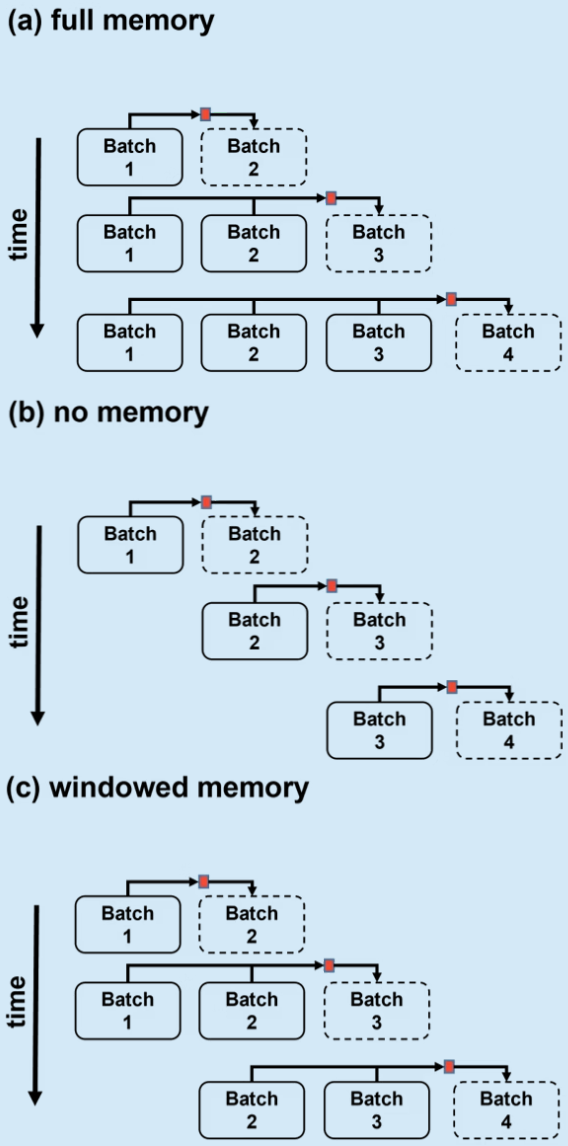
Full-memory

Keeps all observed data points. Preserves a complete history, but tends to be impractical for long or high-dimensional streams.

No-memory

Updated incrementally without storing raw history. Light on memory, but may forget useful patterns or suffer from noisy updates.

The memory policy defines what the model remembers and what it forgets — and directly determines its capacity to adapt.



Windowed memory

Definition

Maintains only a window W of size w of the most recent observations or batches.

> $W_t = \{x_{t-w+1}, ..., x_t\}$

Parameters: w (window size), x_t (current observation).

Trade-off

Large w → more context, more memory. Small w → faster adaptation, less history. The choice of w controls how much the model remembers from the past.

Pseudocode — Window-based Management

```
INPUT: stream X = x₁, x₂, ... , window size w
PRE-CONDITION: w ≥ 1; buffer initialized with available history

STEP 1 — Upon receiving xₜ:
        W_{t-1} ← buffer
STEP 2 — Train or update model M using W_{t-1}
STEP 3 — Detect current point:
        δₜ ← M.detect(xₜ)
STEP 4 — Update memory:
        buffer.append(xₜ)
        if len(buffer) > w:
            buffer.remove_first()
STEP 5 — Emit δₜ

OUTPUT: sequence of decisions δ₁, δ₂, ...
POST-CONDITION: δₜ uses only information available before or at
            instant t, without training on the point itself
            before the decision
```

Full-memory, no-memory, and windowed memory define different strategies for retaining historical information.

Retraining with Recent Batches

Definition

The model is periodically rebuilt using only the B most recent batches, discarding older data.

> $M_{t+1} \leftarrow$ treinar($\{B_{t-}B_{+1}, ..., B_t\}$)

Parameters: B (number of retained batches), B_t (current batch), retraining period r.

When to use

Useful when drift is abrupt and the old model quickly becomes obsolete. Computational cost is higher than incremental learning.

Pseudocode — Retraining with Recent Batches

ENTRADA: fluxo em batches B_1, B_2, ..., janela B, período r
PRÉ-CONDIÇÃO: M_0 treinado, fila de batches inicializada vazia

PASSO 1 — Ao receber batch B_t:
    fila.append(B_t)
    se len(fila) > B:
        fila.remove_first()
PASSO 2 — Se t mod r = 0 (período de retreinamento):
        $M_{t+1} \leftarrow$ treinar(fila)
    Senão:
        $M_{t+1} \leftarrow M_t$
PASSO 3 — Para cada $x_t \in B_t$:
    $\delta_t \leftarrow M_{t+1}$.detectar($x_t$)
    emitir δ_t
PASSO 4 — $t \leftarrow t + 1$; voltar ao PASSO 1

SAÍDA: sequência δ_1, δ_2, ... e modelo retreinado periodicamente
PÓS-CONDIÇÃO: modelo usa apenas os B batches mais recentes

Incremental Learning

Definition

The model M_t is updated with each new example or batch, producing M_{t+1} without revisiting the full history.

> $M_{t+1} \leftarrow \text{update}(M_t, x_t)$

Parameters: M_t (current model), x_t (new observation), η (learning rate).

Challenges

Noise can degrade the model at each step. Real drift must be absorbed, but noise should not. Catastrophic forgetting can erase useful patterns.

Pseudocode — Incremental Learning

ENTRADA: fluxo $X = x_1, x_2, \ldots$, modelo inicial M_0, taxa η
PRÉ-CONDIÇÃO: M_0 treinado em dados históricos ou aleatório

PASSO 1 — Ao receber x_t:
    $\delta_t \leftarrow M_t.\text{detectar}(x_t)$
PASSO 2 — Calcular gradiente ou estatística de atualização:
    $\Delta \leftarrow \text{gradiente}(M_t, x_t)$
PASSO 3 — Atualizar modelo:
    $M_{t+1} \leftarrow M_t - \eta \cdot \Delta$
PASSO 4 — Emitir δ_t
PASSO 5 — $t \leftarrow t + 1$; $M_t \leftarrow M_{t+1}$; voltar ao PASSO 1

SAÍDA: sequência $\delta_1, \delta_2, \ldots$ e modelo atualizado M_t
PÓS-CONDIÇÃO: M_t reflete os dados mais recentes sem rever histórico

Non-stationarity and concept drift

Non-stationarity

The generating process can change over time. What was typical yesterday may no longer be typical today. This change directly affects detectors trained in the past.

Concept drift — formal definition

Concept drift occurs when the joint distribution P(X, y) changes over time in the data stream.

> $\exists t_1 < t_2 : P_{t_1}(X, y) \neq P_{t_2}(X, y)$

This brings online detection closer to change point detection and model adaptation.

Types of drift

Abrupt

P changes instantaneously at t*. Requires immediate retraining.

Gradual

P transitions smoothly. Incremental learning with small λ is appropriate.

Recurring

P returns to a previous state. Models with memory of past concepts are advantageous.

Stability-Plasticity Dilemma

Definition

Every online system faces the tension between resisting irrelevant variations (stability) and adapting quickly to real changes (plasticity).

> equilibrium: α·stability + (1−α)·plasticity

Parameter $\alpha \in [0,1]$ controls the balance. $\alpha \to 1$: rigid system. $\alpha \to 0$: highly adaptive system.

Consequences of Imbalance

Excess stability: drift is not absorbed, model ages. Excess plasticity: noise and rare events corrupt the model.

Practical Manifestations

Learning rate η

High $\eta \to$ plasticity. Low $\eta \to$ stability. Adaptive schedules adjust η dynamically.

Window size w

Small $w \to$ fast adaptation. Large $w \to$ long memory and resistance to noise.

Drift threshold α

Low $\alpha \to$ detects drift easily (more plastic). High $\alpha \to$ requires strong evidence (more stable).

Regularization

Penalizing large changes in the model favors stability without sacrificing all plasticity.

Active learning for drift

Definition

The system detects drift before adapting the model. The update only occurs when there is statistical evidence of a concept change.

> if drift_detected(x_t) → M_{t+1} ← retrain(M_t, W_t)

Parameters: drift detector D_drift, threshold α, reference window W_ref.

Advantage and risk

Reduces unnecessary adaptations caused by noise. Depends on the quality of the drift detector: false negatives delay adaptation.

Pseudocode — Active drift adaptation

INPUT: stream X, model M_0, drift detector D_drift, threshold α
PRE-CONDITION: M_0 trained, W_ref initialized with historical data

STEP 1 — On receiving x_t:
    $\delta_t \leftarrow M_t$.detect(x_t)
STEP 2 — Test for drift:
    p_value ← D_drift.test(W_ref, W_t)
    drift ← (p_value < α)
STEP 3 — If drift = true:
        M_{t+1} ← retrain(M_t, W_t)
        W_ref ← W_t
    Else:
        M_{t+1} ← M_t (no change)
STEP 4 — Emit δ_t
STEP 5 — t ← t + 1; go back to STEP 1

OUTPUT: sequence δ_1, δ_2, … and adapted model
POST-CONDITION: model only changes when drift is confirmed

Passive learning for drift

Definition

The model is updated continuously at every batch or observation, regardless of any drift alarm.

> $M_{t+1} \leftarrow \text{update}(M_t, x_t) \quad \forall t$

Parameters: forgetting rate $\lambda \in (0,1]$, observation weight by age. λ close to 1 preserves long memory. Smaller λ accelerates forgetting. The weight decays with the age of the observation, not with the absolute time of the current observation.

Advantage and risk

Tracks gradual changes without the need for a drift detector. May absorb noise or rare events as normal behavior, degrading specificity.

Pseudocode — Passive drift adaptation

INPUT: stream X, model M_0, forgetting rate λ
PRE-CONDITION: M_0 trained, $\lambda \in (0,1]$

STEP 1 — Upon receiving x_t:
    $\delta_t \leftarrow M_t.\text{detect}(x_t)$
STEP 2 — Assign weights by age:
    for each past observation x_i in history:
        $\text{weight}_i \leftarrow \lambda^{(t - i)}$
    new observation x_t enters with weight 1
STEP 3 — Update model:
    $M_{t+1} \leftarrow \text{update}(M_t, x_t, \text{weight}=1,$
        history weighted by $\text{weight}_i)$
STEP 4 — Emit δ_t
STEP 5 — $t \leftarrow t + 1$; go back to STEP 1

OUTPUT: sequence $\delta_1, \delta_2, \ldots$ and continuously updated model
POST-CONDITION: older observations have weight $\lambda^{(t-i)} \to 0$,
    ensuring gradual forgetting by age

Online prediction

Definition

Goes beyond detecting past or ongoing events. Aims to predict whether an event will occur in a future window [t+1, t+h].

> $P(\text{evento em } [t+1, t+h] \mid x_1, \ldots, x_t)$

Parameters: h (prediction horizon), probability threshold p* for alert emission.

Operational value

Useful when the goal is mitigation, prevention, or early warning. The value of the alert depends on the time available to act before the event.

Approaches

Classification-based

Trains a classifier with event labels. Requires labeled instances and handles class imbalance.

Regression-based

Predicts future values of the series and applies a detector to the forecast. Does not require direct labels.

Probability-based

Estimates P(event | history) directly. Allows calibrating the alert threshold by cost-benefit analysis.

Prediction by Classification

Definition

Extracts sequences and features from previous observations and trains a classifier to predict event labels.

$$\hat{y}_{t+1} = C(\varphi(x_1, ..., x_t))$$

Parameters: C (classifier), φ (feature extraction function), $\hat{y} \in \{0,1\}$ (predicted label).

Requirements

Requires reliable labels to train the classifier. Suitable when events are well-defined and labeling is feasible.

Operational Challenges

1 Scarce labeled instances

Many streams have few labeled instances available.

2 Strong class imbalance

Events are rare: positive/negative ratio can be 1:1000 or worse.

3 Costly and imprecise labeling

Obtaining labels from experts can be slow, expensive, or error-prone.

Prediction by regression

Definition

Uses the history $X = \langle x_1, ..., x_t \rangle$ to predict future values of the series. Event detection is applied over the predicted series.

$$\hat{x}_{t+h} = F(x_1, ..., x_t)$$

Parameters: F (regression model), h (forecast horizon), ε (forecast error).

Advantage

Works without direct event labels. The event is inferred when $\hat{x}_{t+h}$ exceeds a threshold or exhibits an anomalous pattern.

Decision flow

- Receive x_t and update history H.
- Predict: $\hat{x}_{t+1:t+h} \leftarrow F(H)$.
- Apply detector D over $\hat{x}_{t+1:t+h}$.
- Emit alert if D detects an event in the predicted series.

Detecting events in the predicted series

Two-stage approach

First, a future sequence $\hat{X}_{t+1:t+h}$ is predicted using the available history. Then, a detector is applied to the predicted series.

> $\hat{X}_{t+1:t+h} = F(x_1, \ldots, x_t)$

Parameters: F (forecasting model), h (horizon), D (event detector).

Interpretation

$\hat{\delta}_{t+k} = D(\hat{X}_{t+1:t+k})$: event predicted at horizon k. Allows issuing an alert before the actual occurrence.

Pseudocode — Online prediction followed by detection

INPUT: stream X, forecasting model F, detector D, horizon h
PRE-CONDITION: F trained, D calibrated, $h \geq 1$

STEP 1 — Upon receiving x_t:
    update history: $H \leftarrow H \cup \{x_t\}$
STEP 2 — Predict future sequence:
    $\hat{X}_{t+1:t+h} \leftarrow F(H)$
STEP 3 — Apply detector to predicted series:
    for k = 1 to h:
        $\hat{\delta}_{t+k} \leftarrow D(\hat{X}_{t+1:t+k})$
STEP 4 — If $\exists k : \hat{\delta}_{t+k} = 1$:
        issue early alert with horizon k
STEP 5 — $t \leftarrow t + 1$; return to STEP 1

OUTPUT: early alerts with estimated horizon k
POST-CONDITION: alert issued before actual event at t+k

Online multivariate anomaly detection

Definition

Detects anomalies in streams with multiple correlated variables, where the anomaly may lie in the relationships between dimensions, not just in individual values.

> $s_t = \text{anomaly}(x_t \in \mathbb{R}^d, W_t)$

Parameters: d (dimensionality), W_t (multivariate reference window).

Specific challenges

High dimensionality and relationships that change over time. Anomalies may be invisible in each dimension in isolation, but evident in the joint structure.

Approaches

Distance-based

COD, MCOD: distance in $\mathbb{R}^d$ space. Suffers from the curse of dimensionality.

Tree-based

HS-Tree, iForestASD: partitioning of the multivariate space. Efficient and incremental.

Projection-based

LODA, RS-Hash: projection to $\mathbb{R}^m$ with $m \ll d$. Scalable to high dimensions.

Distance-based methods

Principle

Monitor distances between observations in the stream. An observation is anomalous if it is far from its nearest neighbors.

> $s_t = \text{dist}(x_t, \text{kNN}(x_t, W_t))$

Parameters: k (number of neighbors), W_t (reference window), dist (distance metric).

Algorithms

COD (Continuous Outlier Detection): continuously tracks distance-based outliers. MCOD: version with efficient sliding window maintenance.

Operational properties

No distributional assumption

Does not assume a parametric form for P(X). Robust to arbitrary distributions.

Cost per observation

$O(k \cdot |W_t|)$ per step. Index structures (KD-tree, LSH) reduce cost in high dimensions.

Sensitive to dimensionality

In high dimensions, distances concentrate. Projections or dimensionality reduction may be necessary.

Tree-based Methods

Principle

Recursively partition the data space to isolate anomalous observations. Anomalies are isolated with fewer partitions.

> s_t = profundidade_de_isolamento(x_t, T)

Parameters: T (tree structure), expected isolation depth, update window.

Algorithms

HS-Tree (Half-Space Tree): updates a hierarchical half-space structure at each batch. iForestASD: Isolation Forest adapted for data streams with drift.

Operational Properties

Efficiency

Detection cost O(log n) per observation after tree construction.

Incremental Update

HS-Tree updates internal counters without rebuilding the tree at each step.

Interpretability

The isolation path can indicate which dimensions contribute to the anomaly.

Projection-based Methods

Principle

Reduce dimensionality through random or learned projections, detecting anomalies in the projected space.

> $\tilde{x}_t = Px_t, \quad s_t = \text{anomalia}(\tilde{x}_t)$

Parameters: $P \in \mathbb{R}^{m \times d}$ (projection matrix), $m \ll d$ (reduced dimension).

Algorithms

LODA (Lightweight Online Detector of Anomalies): uses histograms on 1D random projections. RS-Hash: combines projections with hashing for high-dimensional streams.

Operational Properties

Scalability

Cost O(m·d) per observation. Suitable for high-dimensional streams where distance-based methods fail.

Lightweight Updates

Histograms and hashes are updated incrementally without revisiting historical data.

Information Loss

The projection may discard relevant dimensions. Multiple independent projections compensate for this limitation.

Latency and Temporal Bias

Latency Definition

Let t^* be the actual time of the event and $\hat{t}$ the time of the first detection. The detection latency is:

$$L = \hat{t} - t^*$$

$L > 0$: late detection. $L < 0$: early detection (possible in prediction). $L = 0$: immediate detection.

Derived Metrics

Mean Time To Detect (MTTD): average of L over all events. Persistent alarms: $\delta_t = 1$ for multiple steps after t^*.

Pseudocode — Latency Calculation

```
INPUT: sequence of real events E = {t*_1, ..., t*_n}
       sequence of detections D = {t̂_1, ..., t̂_m}
PRE-CONDITION: each t̂_i is associated with the nearest event in E

STEP 1 — For each pair (t*_i, t̂_i):
        L_i ← t̂_i − t*_i
STEP 2 — Classify:
        if L_i > 0 → late detection
        if L_i < 0 → early detection
        if L_i = 0 → immediate detection
STEP 3 — Calculate:
        MTTD ← mean(L_1, ..., L_n)
        persistent_alarms ← count runs of δ_t=1 after t̂_i

OUTPUT: vector L, MTTD, persistent alarm count
POST-CONDITION: L allows comparison of detectors by temporal bias
```

Hybrid Methods

Motivation

A time series may simultaneously contain point anomalies, change points, and motifs. Specialized detectors capture different and complementary pieces of evidence.

$$\delta_t = \text{combine}(\delta^1_t, \delta^2_t, \ldots, \delta^k_t)$$

Parameters: δ^i_t (decision of detector i), combination rule (voting, score averaging, ensemble).

Combination Strategies

Majority voting: $\delta_t = 1$ if the majority of detectors agree. Score averaging: $s_t = (1/k)\Sigma s^i_t$, then threshold. Stacking: a meta-model learns to combine the scores.

Hybrid Architecture

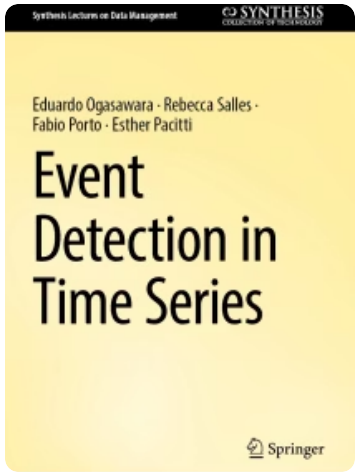
Receive x_t and distribute to k specialized detectors $D^1, \ldots, D^k$.

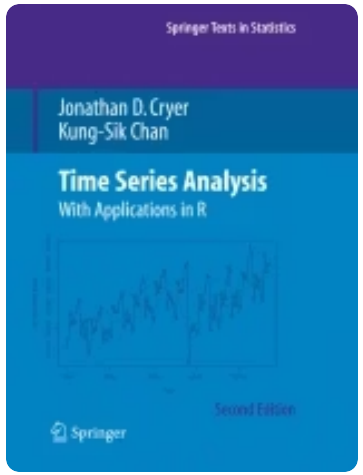
Each D^i independently emits score s^i_t and decision δ^i_t.

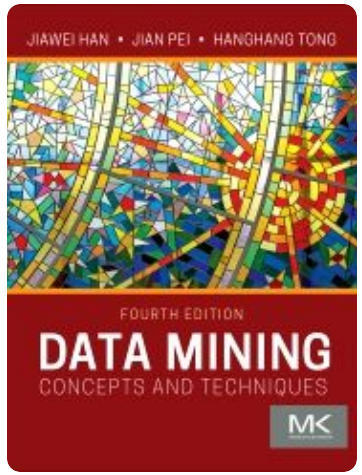
Combination module aggregates $\{s^1_t, \ldots, s^k_t\} \rightarrow s_t$ and applies threshold $\tau \rightarrow \delta_t$.

Emit δ_t with an indication of which detector contributed to the alert.

Bibliographic References







Event Detection in Time Series

Ogasawara, E.; Salles, R.; Porto, F.; Pacitti, E. Springer Nature Switzerland, 2025.

Time Series Analysis: With Applications in R

Cryer, J. D.; Chan, K.-S. Springer, 2008.

Data Mining: Concepts and Techniques

Han, J.; Pei, J.; Tong, H. Morgan Kaufmann, 4th ed., 2022.